MUFBVAR

Version

Table of Contents

Contents:

Welcome to MUFBVAR's documentation!

Introduction

`MUFBVAR` is a module to handel, disaggregate and forecast Multiple frequency data using Bayesian VAR Models.

Installation:

You can use pip to install the module form Gitea: .. code-block:: console

```
foo @ bar :~$ pip install git+https://gitea.efv.admin.ch/efv_fs/MUFBVAR.git
```

MUFBVAR

MUFBVAR package

Module contents

class MUFBVAR. mufbvar_data (*data* , *trans* , *frequencies*) [source]

Bases: `object`

Class to prepare the data that will be used in the MUFBVAR ...

Parameters :

- **data** (*list of pandas DataFrames*) – Data of each frequency stored in a pandas DataFrame, all stored in one list
- **trans** (*list of numpy arrays*) – A separate numpy array for each frequency all stored in a list. /n 0: log is taken 1: divided by 100
- **frequencies** (*List of the frequencies of the data , in order lowest to highest*) – “Y”, “Q”, “M”, “W”, “D”

class MUFBVAR. multifrequency_var (*nsim* , *nburn_perc* , *nlags* , *thining*) [source]

Bases: `object`

MUFBVAR class

Parameters :

- **nsim** (*int*) – Number of simulations
- **nburn_perc** (*numeric*) – Between 0 and 1, proportion of simulations to throw away as burn in.
- **nlags** (*int*) – Number of lags in the highest frequency

- **thining** (*int*) – To save only every nth draw

aggregate (*frequency* , *reset_index = True*)

Aggregates the Mean, Median and quantililes in the highest frequency to the desired frequency.

The Function ensures, that we start at the beginning of a Year or Quarter depending on the chosen frequency

Parameters :

- **frequency** (*str*) – The frequency to which the data should be aggregated to
- **reset_index** (*boolean*) – Schould index be changed to period Index

fanchart (*variables = 'all'* , *save = True* , *name = 'Fancharts'* , *show = True* , *agg = False* , *nhist = 10*)

Creates fan plots of the desired variables.

Parameters :

- **variable** (*list of strings*) – variables for which the plot should be generated, all if it should be generated for all
- **save** (*boolean*) – Whether the plots should be saved. The default is True.
- **name** (*string , optional*) – If the plots should be saved, path/name not including filetype. The default is None.
- **show** (*boolean*) – Whether the plots should be shown. Default is True.
- **agg** (*boolean*) – Whether the aggregated values should be shown
- **nhist** (*int*) – number of historical periods that should be shown on the plot Default is 5

fit (*mufbvar_data* , *hyp*)

Estimates the model using the model parameter specified in the initialization.

And the data provided.

Parameters :

- **mufbvar_data** (*mufbvar_data class object*) – data in the form of a mufbvar_data class object
- **hyp** (*list of list*) –

  list containing list of the hyperparameters for each frequency step

  1. overall tightness
  2. scaling down the variance for the coefficients of a distant lag
  3. number of observations used for obtaining the prior for the covariance matrix of error terms
  4. tuning parameter for coefficients for constant
  5. tuning parameter for the covariance between coefficients

forecast (*H* , *conditionals = None*)

Method to generate the forecasts in the highest frequency.

Parameters :

- **H** (*int*) – Forecast horizon in highest frequnecy
- **conditionals** (*pandas DataFrame or None*) –

  Conditional forecasts

  column names must be the variable names

  no index needed

  either values or np.nan

mean_plot (*variables = 'all'* , *save = True* , *name = 'Output'* , *show = True*)

Creates cumulative mean plots of the forecasts. If the model has converged the cumulative mean should be stable after burnin.

Parameters :

- **variable** (*list of strings*) – variables for which the plot should be generated, all if it should be generated for all
- **save** (*boolean*) – Whether the plots should be saved. The default is True.
- **name** (*string , optional*) – If the plots should be saved, path/name not including filetype. The default is None.
- **show** (*boolean*) – Whether the plots should be shown. Default is True.

save (*filename = 'mufbvar_model.pkl'*)

Saves the MFBVAR Object

Parameters :

filename (*str*) – Path where to save the object. End must be .pkl

scenario_forecast (*H* , *conditionals* , *names* , *agg = True*)

Generates the mean forecasts for multiple scenarios

Parameters :

- **H** (*int*) – Forecast Horizon in highest frequency
- **conditionals** (*list*) – list of panda DataFrames
- **names** (*list*) – list of the names for the scenarios
- **agg** (*boolean*) –

  If true aggregates output to lowest frequency

  Default is True

Returns :

out_dict – Dictionary containing the panda DataFrames for each szenario

Return type :

dict

scenario_plot (*variables = 'all'* , *save = True* , *name = 'Scenario'* , *show = True* , *nhist = 5*)

Creates a plot with the different scenarios

Parameters :

- **scenario_dict** (*dict*) – output of self.scenario_forecast
- **variable** (*list of strings*) – variables for which the plot should be generated, all if it should be generated for all
- **save** (*boolean*) – Whether the plots should be saved. The default is True.
- **name** (*string , optional*) – If the plots should be saved, path/name not including filetype. The default is None.
- **show** (*boolean*) – Whether the plots should be shown. Default is True.
- **agg** (*boolean*) – Whether the aggregated values should be shown
- **nhist** (*int*) – number of historical periods that should be shown on the plot Default is 5

to_excel (*filename* , *agg = False*)

Writes the results to an excel

Parameters :

- **agg** (*Boolean*) – Should the aggregated series be saved
- **filname** (*Sting*) – file path.

update_hyperparameters (*mufbvar_data* , *pbounds* , *init_points* , *n_iter* , *nsim* , *save = False* , *name = 'hyp.txt'*)

This method uses bayesian optimization to find the hyperparameters with the highest mdd

lambda 1: overall tightness

lambda 2: scaling down the variance for the coefficients of a distant lag

lambda 3: number of observations used for obtaining the prior for the covariance matrix of error terms, fixed to 1

lambda 4: . tuning parameter for coefficients for constant

lambda 5: tuning parameter for the covariance between coefficients

Parameters :

- **mufbvar_data** (*mufbvar_data class object*) – data in the form of a mufbvar_data class object
- **pbound** (*dict*) –

  boundries for each hyperparameter

  two frequencies: lambda1_1, lambda2_1, lambda4_1, lambda5_1 three frequencies: lambda1_1, lambda2_1, lambda4_1, lambda5_1, lambda1_2, lambda2_2, lambda4_2, lambda5_2 four frequencies: lambda1_1, lambda2_1, lambda4_1, lambda5_1, lambda1_2, lambda2_2, lambda4_2, lambda5_2, lambda1_3, lambda2_3, lambda4_3, lambda5_3
- **init_points** (*int*) – How many steps of random exploration you want to perform
- **n_iter** (*int*) – How many steps of bayesian optimization you want to perform
- **nsim** (*int*) – number of draws in each MUFBVAR estimation
- **save** (*boolean*) – True if you want to save the hyperparameters as a txt
- **name** (*str*) – path where you want to save the hyperparameters

Returns :

hyp – list containing the optimized hyperparameters

Return type :

list

Examples

Example in python:

```
import MUFBVAR
import pandas as pd
import numpy as np


io_data = "hist.xlsx"

#Model Specification
H = 96        # forecast horizon
nsim     = 100  # number of draws from Posterior Density
nburn    = 0.5  # number of draws to discard
nlags = [6,4]
thining = 1

hyp = (0.09, 4.3, 1, 2.7, 4.3)

frequencies = ["Q","M","W"]



# Load the data
data = []
for freq in range(len(frequencies)):
            freq = frequencies[freq]
            data_temp = pd.read_excel(io_data, sheet_name = freq,
index_col = 0)
            data.append(data_temp)
#Transformations
trans = [np.array((1)), np.array((1,1,1)), np.array((1,1,1,1))]


#Initialize data class
data_in = MUFBVAR.mufbvar_data(data, trans, frequencies)


#Initialize model class
model =  MUFBVAR.multifrequency_var(nsim, nburn, nlags ,thining)

#Estimate the model
model.fit(data_in, hyp = hyp)
```

```
#Conditional forecasts
conditionals = pd.DataFrame({'w_1' : [0.018, 0.025, np.nan, np.nan,
0.0228, 0.05],
                             'm_2' : [ np.nan, 0.002, 0.01 , 0.01,
np.nan, np.nan]})

#Create forecasts in highest frequency
model.forecast(H, conditionals)

#Aggregate
model.aggregate(frequency = "Q")

#Save results
#model.to_excel('out_test.xlsx', agg = True)

#Plots
model.mean_plot(variables = "all", save = False, show = True)

model.fanchart(variables = "all", save = False, show = True, agg =
True, nhist = 10)

# Optimizing Hyperparameters

pbounds = {'lambda1_1': (0.001, 20), 'lambda2_1': (0.01, 10),
'lambda4_1': (0.01, 10), 'lambda5_1': (0.01, 10), 'lambda1_2':
(0.001, 20), 'lambda2_2': (0.01, 10), 'lambda4_2': (0.01, 10),
'lambda5_2': (0.01, 10)}
init_points = 3
n_iter = 8
nsim = 100

hyp = model.update_hyperparameters(data_in, pbounds, init_points,
n_iter, nsim, save = False, name = "hyp.txt")

# scenario analysis

conditionals = [pd.DataFrame({'w_1' : [0.018, 0.025, np.nan, np.nan,
0.0228, 0.05],
                             'm_2' : [ 0.3, 0.002, 0.01 , 0.01,
np.nan, np.nan]}),
                    pd.DataFrame({'w_1' : [-0.02, -0.25, np.nan,
np.nan, -0.228, 0.1],
                             'm_2' : [ -0.2, -0.012, 0 , 0.1, np.nan,
np.nan]}),
                             None]

names = ["good", "bad", "base"]

out_scenarios = scenario_forecast(H, conditionals, names, agg = True)

# Scenario Plot
```

```
scenario_plot(out_scenarios, variables = "all", save = False, name =
"Scenario", show = True, nhist = 10)
```

Example in R:

The module can also be used in r, using the reticulate package:

```
library(reticulate)

setwd(dirname(rstudioapi::getActiveDocumentContext()$path))

mufbvar <- import("MUFBVAR")
pd <- impoty("pandas")
np <- import("numpy")

io_data <- "hist.xlsx"


H <- 96L        # forecast horizon
nsim <- 20000L  # number of draws from Posterior Density
nburn <- 0.5  # number of draws to discard
nlags <- list(6L,4L)
thining = 1

hyp <- c(0.09, 4.3, 1, 2.7, 4.3)

frequencies <- list("Q", "M", "W")


data <- list()
for (freq in 1:length(frequencies)) {
    freq <- frequencies[[freq]]
    data_temp <- pd$read_excel(io_data, sheet_name = freq, index_col
= 0)
    data <- append(data, list(data_temp))
}

#Transformations
trans <- list(np$array((1)), np$array((1,1,1)), np$array((1,1,1,1)))


#Initialize data class
data_in <- mufbvar$mufbvar_data(data, trans, frequencies)


#Initialize model class
model <-  mufbvar$multifrequency_var(nsim, nburn, nlags ,thining)
```

```
#Estimate the model
model$fit(data_in, hyp = hyp)

#Conditional forecasts
conditionals <- pd.DataFrame({'w_1' : [0.018, 0.025, np.nan, np.nan,
0.0228, 0.05],
                              'm_2' : [ np.nan, 0.002, 0.01 , 0.01,
np.nan, np.nan]})

#Create forecasts in highest frequency
model$forecast(H, conditionals)

#Aggregate
model$aggregate(frequency = "Q")

#Save results
#model$to_excel('out_test.xlsx', agg = True)

#Plots
model$mean_plot(variables = "all", save = False, show = True)

model$fanchart(variables = "all", save = False, show = True, agg =
True, nhist = 10L)
```

Indices and tables

- Index
- Module Index